

[17] Spider 2.0: Evaluating Language Models on Real-World Enterprise Text-to-SQL Workflows

저자: Fangyu Lei, Jixuan Chen, Yuxiao Ye 외 다수 **학회/년도:** arXiv:2411.07763, 2024 (ICLR 2025 Oral 채택)
분량: 약 30페이지 + 부록 **역할군:** (A) VAQ 동기 부여

요약

Spider 2.0은 **실세계 기업 환경에서의 Text-to-SQL 복잡성**을 처음으로 체계적으로 측정한 벤치마크이다. 기존 Spider 1.0이 학술 환경에 맞춰진 상대적으로 간단한 쿼리만 다루었다면, Spider 2.0은 **실제 기업 데이터베이스의 극단적 복잡성**을 반영한다.

핵심 발견: 1. **학술 vs. 현실의 거대한 격차:** 모든 LLM이 Spider 1.0에서는 85~91% 정확도를 보이지만, Spider 2.0에서는 15~21%로 급락 (50~70 포인트 격차) 2. **실제 데이터베이스의 극단적 복잡성:** 수천 개 테이블, 1,000+ 컬럼, 100줄 이상의 멀티 스텝 쿼리 3. **VAQ의 현실적 복잡성:** 벡터 검색이 포함되면, 이런 복잡한 쿼리의 최적화는 더욱 어려워짐 4. **ICLR 2025 Oral 채택:** 학계의 큰 주목을 받아 커뮤니티에 강력한 메시지 전달

Spider 2.0은 Exqutor의 동기부여 논문로서, "단순한 벡터 검색을 넘어 실무 복잡도의 VAQ가 필요하다"는 것을 강력히 보여준다.

상세분석

17.1 문제: 학술 벤치마크의 치명적 한계

Spider 1.0의 문제점:

기존 Spider 1.0 벤치마크는 학술 환경의 편의성을 위해 다음과 같이 단순화되어 있었다:

특성	Spider 1.0	현실 기업 데이터베이스
테이블 수	평균 5~10개	수백~수천 개 (대기업은 1만+ 테이블)
컬럼 수	평균 ~20개	1,000개 이상 (한 테이블만 500개 컬럼)
SQL 방언	SQLite만	BigQuery, Snowflake, PostgreSQL, T-SQL, Oracle 등 다양
쿼리 복잡도	단일, 간단한 쿼리	100줄 이상의 멀티 쿼리 워크플로우
전처리/후처리	불필요	CRITICAL: CLI 호출, 파일 변환, API 통합
도메인 지식	기본 SQL	도메인별 비즈니스 로직 필수

실제 예시 쿼리:

```

-- 실제 기업 쿼리 (Spider 2.0)
WITH sales_data AS (
  SELECT
    order_id,
    customer_id,
    DATE_TRUNC('month', order_date) AS month,
    SUM(amount) OVER (PARTITION BY customer_id ORDER BY order_date
                      ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW) AS cumsum
  FROM orders
  WHERE order_status = 'completed'
),
customer_segments AS (
  SELECT
    customer_id,
    CASE
      WHEN cumsum > 100000 THEN 'VIP'
      WHEN cumsum > 50000 THEN 'Premium'
      ELSE 'Standard'
    END AS segment
  FROM sales_data
),
geographic_analysis AS (
  SELECT
    c.customer_id,
    cs.segment,
    COUNT(DISTINCT s.order_id) AS order_count,
    AVG(s.amount) AS avg_order_value,
    g.region,
    g.country
  FROM customer_segments cs
  JOIN customers c ON cs.customer_id = c.id
  JOIN sales_data s ON c.id = s.customer_id
  JOIN geographic_data g ON c.zip_code = g.zip_code
  GROUP BY c.customer_id, cs.segment, g.region, g.country
)
SELECT * FROM geographic_analysis
WHERE order_count > 10 AND region = 'APAC'
ORDER BY avg_order_value DESC;

```

Spider 1.0은 이렇게 복잡한 쿼리를 전혀 다루지 않았다.

17.2 Spider 2.0의 구성

632개의 실세계 Text-to-SQL 문제:

각 문제는: - 실제 기업 데이터베이스 스키마 (Kaggle, real company datasets) - 자연어 질문: 분석가나 의사결정자가 실제로 할 만한 질문 - 정답 SQL 워크플로우: 1명 이상의 SQL 전문가가 작성·검증 - 멀티 스텝 프로세스:

쿼리 → 결과 내보내기 → 파이썬/R 후처리 → 최종 답변

데이터셋의 다양성:

- **도메인**: 금융, 의료, 전자상거래, 물류, HR, 마케팅 등 15+ 산업
- **데이터 크기**: 소규모 ~ 100억 행
- **스키마 복잡도**: 최소 10 테이블 ~ 최대 1,000+ 테이블

17.3 성능 결과: 학술 vs. 현실의 극심한 격차

주요 발견:

모델	Spider 1.0 정확도	Spider 2.0 정확도	격차	성능 저하
o1-preview (OpenAI)	91.2%	21.3%	-69.9 pp	76.6% 저하
GPT-4o	87.5%	17.0%	-70.5 pp	80.6% 저하
Claude 3.5 Sonnet	85.3%	15.4%	-69.9 pp	81.9% 저하
Gemini 2.0	84.1%	16.2%	-67.9 pp	80.7% 저하

격차의 의미:

- o1-preview는 Spider 1.0에서는 SOTA(최고 성능)이지만, Spider 2.0에서는 **5명 중 4명이 실패**한다.
- 이는 단순한 "개선 여지"가 아니라, **패러다임의 전환**이 필요함을 의미한다.

17.4 성능 저하의 주요 원인 분석

1. 스키마 복잡성:

- 테이블 400개, 컬럼 8,000개 이상인 실제 데이터베이스
- LLM의 context window에 전체 스키마를 넣을 수 없음
- **적절한 테이블·컬럼 선택이 가장 어려운 부분** (쿼리 생성보다도)

2. 비표준 SQL 방안:

```
-- BigQuery 방언
SELECT ARRAY_AGG(STRUCT(col1, col2))
FROM table1
WHERE DATE BETWEEN @start_date AND @end_date;

-- 동일한 의미를 T-SQL로
SELECT col1, col2
INTO #temp_result
FROM table1
WHERE date_column BETWEEN @start_date AND @end_date;
```

방언이 다르면 문법이 완전히 달라지므로, LLM도 혼동한다.

3. 멀티 스텝 워크플로우:

단순 SQL 쿼리뿐 아니라:

```
Step 1: SQL 쿼리 실행
Step 2: CSV로 내보내기
Step 3: 파이썬으로 데이터 정제
Step 4: 그래프 생성
Step 5: 비즈니스 로직 적용
```

LLM이 모든 단계를 정확히 이해하고 조율해야 한다.

4. 도메인 지식 부재:

```
-- 일반 지식으로는 불가능:
SELECT revenue, cost
FROM sales
WHERE 상품_ID IN (SELECT ID FROM 상품 WHERE 카테고리 = '핵심전략상품')
```

"핵심전략상품"은 회사 내부에서만 정의된 개념이다. 회사 특정 용어를 모르면 불가능하다.

17.5 상세한 분류 및 오류 분석

오류 유형:

1. **스키마 선택 오류** (40%): 잘못된 테이블·컬럼 선택
2. **논리 오류** (25%): 올바른 테이블을 선택했지만 논리가 틀림
3. **방언 오류** (20%): SQL 문법 오류 (방언별)
4. **조인 조건 오류** (10%): JOIN ON 조건을 잘못 지정
5. **기타** (5%): 정렬, 그룹화, 윈도우 함수 등

패턴:

- 간단한 쿼리(1~2 테이블, 단순 필터): 90% 정확도
- 중간 복잡도(5~10 테이블, 2~3 JOIN): 40~50% 정확도
- 매우 복잡(100+ 테이블, 복잡한 로직): 5~10% 정확도

17.6 ICLR 2025 Oral 채택의 의미

Spider 2.0이 ICLR 2025의 **Oral 세션**에 채택된 것은:

1. **커뮤니티의 강한 관심**: Text-to-SQL의 현실적 도전이 얼마나 심각한지 알려줌
2. **새로운 연구 방향 제시**: 단순한 쿼리 생성을 넘어, 스키마 이해, 도메인 지식, 멀티 스텝 추론이 중요함을 강조
3. **LLM 한계의 명확화**: "현재의 LLM만으로는 실무 Text-to-SQL을 해결할 수 없다"는 증거 제시

17.7 본 논문과의 관계: VAQ의 극단적 복잡성

Spider 2.0과 VAQ의 연결:

LLM이 Spider 2.0의 복잡한 쿼리를 생성할 때, 만약 벡터 검색이 포함되면 **VAQ**가 된다.

```
-- Spider 2.0 스타일의 복잡 쿼리에 벡터 검색 추가
WITH customer_vectors AS (
  SELECT
    customer_id,
    embedding,
    purchase_history_summary
  FROM customer_embeddings
  WHERE embedding <=> target_vector < 0.1 -- 벡터 검색
),
enriched_analysis AS (
  SELECT
    cv.customer_id,
    cv.purchase_history_summary,
    c.segment,
    ...100줄 이상의 복잡한 쿼리...
  FROM customer_vectors cv
  JOIN customers c ON cv.customer_id = c.id
  JOIN orders o ON c.id = o.customer_id
  ... 수십 개 테이블 JOIN ...
)
SELECT * FROM enriched_analysis;
```

이제 옵티마이저의 과제:

1. **벡터 검색의 선택도 추정**: 얼마나 많은 고객이 target_vector에 유사한가?

2. **JOIN 순서 결정:** 벡터 검색 결과부터 시작할까, 아니면 다른 필터부터?
3. **인덱스 사용 전략:** HNSW 인덱스를 사용할까, 아니면 Sequential Scan?

고정 33.3% 선택도로는 절대 불가능하다.

17.8 Spider 2.0의 함의와 Exqutor의 가치

Spider 2.0이 보여주는 것:

1. Text-to-SQL의 진정한 어려움은 쿼리 문법이 아니라 의미 이해와 스키마 선택
2. 멀티 스텝 워크플로우와 도메인 지식의 중요성
3. 실무 환경의 극단적 복잡도는 학술 벤치마크로 포착 불가

Exqutor의 역할:

LLM이 생성한 복잡한 VAQ를 효율적으로 실행하려면:

LLM 생성 SQL (극도로 복잡)

- PostgreSQL/DuckDB 옵티마이저
- Exqutor ECQO (벡터 카디널리티 추정)
- 정확한 실행 계획
- 빠른 결과

만약 옵티마이저가 벡터 검색의 선택도를 부정확하게 추정하면, 수백ms 이상의 성능 저하가 불가피하다.

17.9 향후 연구 방향

Spider 2.0을 기반으로 한 향후 연구:

1. **Spider 3.0 (가설):**
2. 벡터 검색이 명시적으로 포함된 문제
3. RAG-like 멀티 스텝 검색 + SQL 분석
4. 실무 Text-to-RAG-to-SQL 워크플로우
5. **Exqutor + Spider 2.0 통합 벤치마크:**
6. TPC-H/TPC-DS의 벡터 검색 확장
7. Spider 2.0의 실무 복잡도와 결합
8. "현실적인 VAQ 성능 평가"
9. **LLM 기반 스키마 이해:**
10. 대규모 데이터베이스에서 자동으로 관련 테이블·컬럼 선택
11. 도메인 지식 자동 학습

추가 제기 문제

1. **Spider 2.0의 난이도 설정**: 정확도가 15~21%라는 것이 "합리적인 도전"인가, 아니면 "너무 어려워서 무의미"한가에 대한 커뮤니티 논의가 필요하다.
2. **평가 메트릭의 재고**: Exact Match 정확도만 측정하는데, 실제로는 "부분적으로 올바른 쿼리"도 가치가 있을 수 있다. 예를 들어, 절반의 조인이 올바르면 결과의 50%는 맞을 것이다. 더 세분화된 평가 메트릭이 필요하다.
3. **도메인 적응(Domain Adaptation)**: 한 회사의 데이터베이스에서 학습한 LLM이 다른 회사의 데이터베이스에 얼마나 잘 일반화되는가? Transfer learning의 가능성과 한계는?
4. **멀티모달 Text-to-SQL**: 비정형 데이터(이미지, 문서)를 포함한 벡터 검색이 추가되면, Text-to-SQL의 복잡도는 또 몇 배로 증가할 것인가?